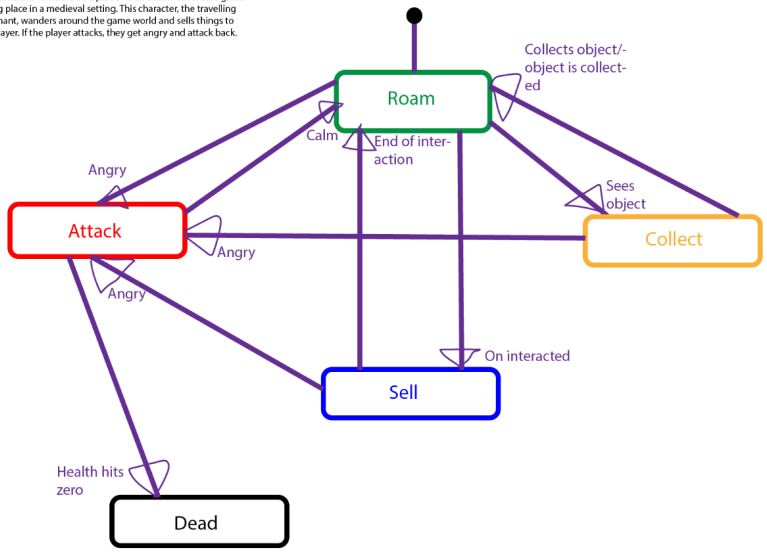
Finite State Machine #1: Travelling Merchant

The game this AI would be a part of is an action-adventure game taking place in a medieval setting. This character, the travelling merchant, wanders around the game world and sells things to the player. If the player attacks, they get angry and attack back.



State 1: Roam

The merchant wanders around the game world along a set path.

State 2: Sell

The merchant converses with another entity. In this state, the merchant can aquire/lose items or aquire/lose money. The player can directly control this when they interact with the merchant, but when other entities interact, it's random.

State 3: Attack

The merchant pulls out a sword and chases the target around. When within a certain range of the player, the merchant will strike with the sword, dealing damage.

State 4: Collect

When the merchant sees a valuable object in the level, he'll run over to that spot to pick it up. While in this state, attempting to talk to him will be impossible.

State 4: Dead

The merchant ragdolls and drops some loot. The merchant cannot be brought back to life.

Feedback state: Angry

When an entity hits the merchant with an attack of some kind, the merchant's eyes will glow red and he'll shout "have at thee!" The attacker becomes the target.

Feedback state: Calm

When the target exits the merchant's vision cone for three minutes, his eyes will return to normal.

Feedback state: Health hits zero

When the merchant gets hit enough times to the point of having zero health, the merchant will fall over dramatically and drop his backpack.

List of sensors

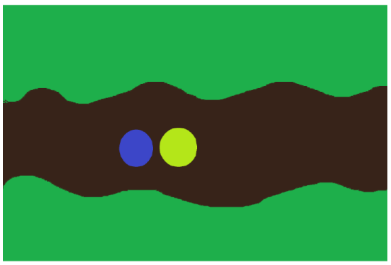
- Vision cone that detects entities. When in attack, is used to detect target and chase after them. When in roam, is used to go into collect when a valuable object is in range.

- Audio range that detects noise near the merchant. in attack is used to listen for target. When hears target, turn towards them.

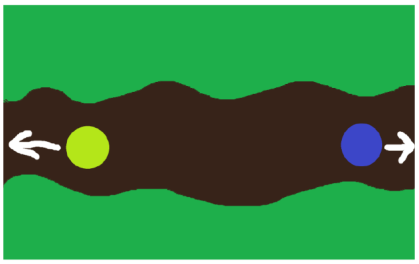
- Hitbox on body. In roam and sell is used to detect when an attack is used on the merchant. If an attack is used, first enter angry state, then enter attack state.

Gameplay Scenario

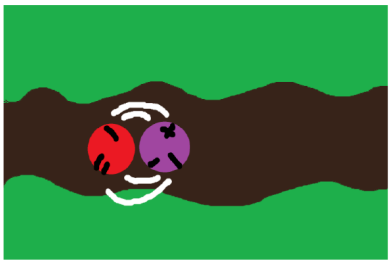
1.



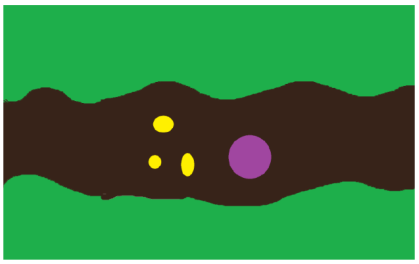
2.



3.



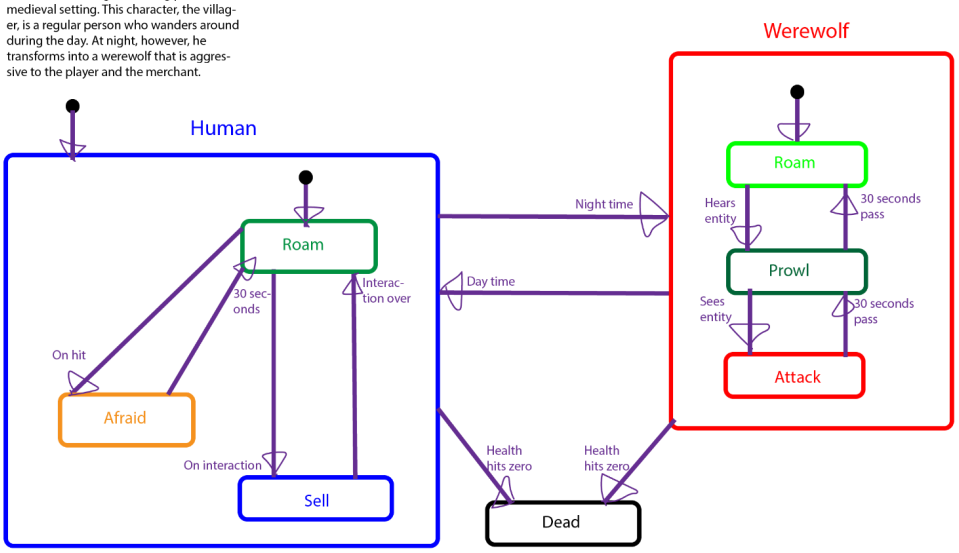
4.



In this scenario, the merchant first sells the villager some valuable items, perhaps that the player cannot afford at the moment. Then, after they part ways, the sun goes down and the villager transforms. The werewolf attacks the player, the player wins, and then the player aquires good loot (indirectly caused by the merchant).

Finite State Machine #2: Villager

The game this AI would be a part of is an action-adventure game taking place in a medieval setting. This character, the villager, is a regular person who wanders around during the day. At night, however, he transforms into a werewolf that is aggressive to the player and the merchant.



State 1: Roam

The villager wanders around the game world along a set path.

State 2: Sell

The villager converses with another entity. In this state, the villager can aquire/lose items or aquire/lose money. The villager cannot interact with the player or other villagers. Only merchants.

State 3: Afraid

The villager runs away in a random direction from their attacker until the attacker is out of range for long enough.

State 4: Dead

The villager ragdolls and drops some loot. The villager cannot be brought back to life. The loot the villager drops is determined by the dealings made with the merchant.

Feedback state: Gets hit
When the villager gets attacked in human form, he'll pause, scream, and then enter afraid

Feedback state: Sees entity
When the villager is in werewolf form and he sees either a player or a merchant, he'll pause, an exclamation point will pop up above his head, and he'll let out a howl.

Feedback state: Health hits zero
When the villager gets hit enough times to the point of having zero health, the villager will fall over dramatically and drop whatever items he is currently holding.

List of sensors

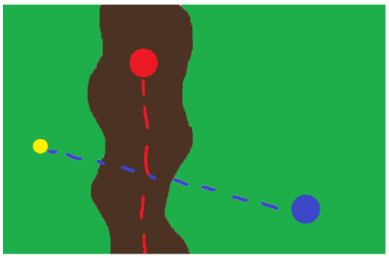
- Vision cone that detects entities. When in werewolf in prowl or roam, when a player or merchant enters vision cone, enter attack.

- Audio range that detects noise near the villager. In werewolf state, is used when in roam state to detect players and merchants. When in range, go into prowl.

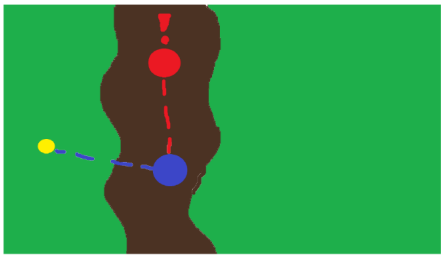
- Hitbox on body. In roam and sell is used to detect when an attack is used on the merchant. If an attack is used, first enter

Gameplay Scenario

1.



2.



3.

4.

In this scenario, the player first drops a valuable item near the werewolf's path. Then, the merchant naturally sees the item (vision cone) and goes to pick it up. Then, the werewolf sees the merchant (vision cone) and goes to attack. The merchant and werewolf fight for a little bit, and the werewolf wins thanks to higher HP and attack power. The merchant drops some loot, the werewolf runs off and the player is free to pick up the loot.

FSM Design Justification

The reason I designed these AI states in this way is so that depending on a day/night cycle, the behaviors of the objects would change, and so would the relationships between those objects. I wanted the player to ultimately exploit these changes to get benefits, therefore creating a satisfying game loop where the player feels that they are cheating the system by utilizing these changing behaviors. For example, say the player wants to purchase an item. The item is too expensive. The player could try to fight the merchant, but that could result in the player getting killed. Instead, the player realizes that they can exploit the specific “collect” behavior the merchant has in order to activate the spatial relationship between the merchant and the werewolf. Placing a high value object on the werewolf’s path will make them fight, and I specifically gave the werewolf higher health and attack power as an attribute so that without help, the merchant would most likely lose the fight and the player would be able to take the loot afterwards. I designed the feedback loops to prompt the players with a familiar mental model. For example, when a player gets seen by a werewolf, the animation of an exclamation point above his head and the howl are meant to tell the player the werewolf is getting ready to attack, and the expected response is for the player to run away or fight (the action). If the player runs out of the vision cone of the werewolf for long enough, the AI will receive that feedback and gradually return to the roaming behavior, then updating the player’s mental model. If the player attacks and kills the werewolf, the AI will receive that feedback, drop dead and drop whatever the villager was carrying, updating the player’s mental model.